



Laboratório 5: Serviços Web REST em Java

18/05/2026

Sumário

1 Criando estrutura do projeto Java Gradle com o framework Spring	1
2 Primeiro exemplo: Uso do verbo GET HTTP	2
2.1 Crie os pacotes e as classes Java	2
2.2 Como executar o projeto	3
2.3 Como consumir o serviço usando o cURL	3
3 Segundo exemplo: Verbos GET, POST, PUT e DELETE	4
3.1 Crie os pacotes e as classes Java	4
3.2 Como consumir o serviço usando o cURL	8
4 Aplicação Java para consumir serviço REST	8
5 Documentação de API com OpenAPI 3	10

1 Criando estrutura do projeto Java Gradle com o framework Spring

Java possui diversos *frameworks* e *APIs* que possibilitam o desenvolvimento de Serviços Web RESTful, como a API Java para RESTful Web Services (JAX-RS) e o framework Spring¹. Nesse laboratório faremos uso do Spring Boot para criar o projeto. Siga os passos abaixo (também é possível fazer pelo VSCode + extensão Spring Boot Extension Pack ou IntelliJ Ultimate):

1. Acesse <https://start.spring.io/>
2. Marque as seguintes opções:
 - **Project:** Gradle - Groovy
 - **Language:** Java
 - **Spring Boot:** 4.0.6
3. Em **project metadata** preencha com os seguintes valores:
 - **Group:** engtelecom.std
 - **Artifact:** labrest
 - **Description:** Laboratório RESTful com Java
 - **Packing:** Jar
4. Em **dependencies** clique no botão **Add dependencies**
 - Adicione Spring Web
5. Clique no botão **Generate** para baixar o arquivo ZIP contendo o projeto
6. Descompacte o ZIP, pelo terminal entre neste diretório e atualize a versão do Gradle Wrapper:

¹<https://spring.io/>

```
cd labrest
gradle wrapper --gradle-version latest
```

7. Por fim, abra esse projeto na IDE IntelliJ ou com o Visual Studio Code.

Edite o arquivo `src/main/resources/application.properties` e adicione as seguintes linhas para diminuir o número de mensagens de LOG que aparecem ao executar a aplicação.

```
spring.main.banner-mode=off

# TRACE, DEBUG, INFO, WARN, ERROR, FATAL, OFF
# Nível de log para o Spring Framework
logging.level.root=ERROR
logging.level.org.springframework.web=ERROR

# Nível de log para as minhas classes
logging.level.engtelecom.std.labrest=WARN
```

2 Primeiro exemplo: Uso do verbo GET HTTP

Na Tabela 1 é apresentada a API desse primeiro exemplo que teve como base o guia disponível em <https://spring.io/guides/gs/rest-service/>. São descritos os verbos HTTP que poderão ser usados sobre cada recurso, bem como a informação que será retornada.

Tabela 1: API do Serviço Web de Saudação

Método	Recurso	Resposta
GET	/saudacao	Documento JSON {"id":1,"nome":"Olá Mundo"}
	/saudacao?nome=SeuNome	Documento JSON {"id":2,"nome":"Olá SeuNome"}

2.1 Crie os pacotes e as classes Java

Crie os pacotes `entities` e `controller` e as classes `Saudacao.java` e `SaudacaoController.java` para ficar de acordo com a estrutura apresentada abaixo:

```
.
|-- engtelecom
    |-- std
        |-- labrest
            |-- LabrestApplication.java
            |-- controller
            |   |-- SaudacaoController.java
            |-- entities
            |-- Saudacao.java
```

Na Listagem 1 é apresentado o conteúdo da classe *Record*² `Saudacao.java`. Sempre que o recurso `/saudacao` for consumido com o método GET, será criada uma instância dessa classe e os atributos da mesma serão convertidos para um documento JSON e retornados ao solicitante.

Listagem 1: `Saudacao.java`

```
package engtelecom.std.labrest.entities;

public record Saudacao(long id, String nome){}
```

²É um tipo especial de classe ideal quando deseja-se criar objetos imutáveis e usados para transferência de dados e evitando assim a ter que recorrer a soluções com o Projeto Lombok para evitar ter que criar métodos *getters*, construtores, etc. Veja mais em <https://docs.oracle.com/en/java/javase/17/language/records.html>

Na Listagem 2 é apresentado o conteúdo do código da classe responsável por processar os pedidos e instanciar objetos da classe `Saudacao.java`. Como apresentado na Tabela 1, o recurso `/saudacao` tem um parâmetro `nome` que é opcional.

Listagem 2: `SaudacaoController.java`

```
package engtelecom.std.labrest.controller;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;
import java.util.concurrent.atomic.AtomicLong;
import engtelecom.std.labrest.entities.Saudacao;

@RestController
public class SaudacaoController {
    private static final String MENSAGEM = "Olá %s";
    private final AtomicLong contador = new AtomicLong();

    @GetMapping("/saudacao")
    public Saudacao saudacao(@RequestParam(value = "nome", defaultValue = "mundo") String nome){
        return new Saudacao(contador.incrementAndGet(), String.format(MENSAGEM, nome));
    }
}
```

2.2 Como executar o projeto

Com as classes finalizadas, você poderá executar o projeto usando o gradle. Execute a tarefa do gradle (*task*) chamada `bootRun` que, no menu da IDE, estará dentro da categoria `application`. Se optar por executar no terminal:

```
./gradlew bootRun
```

Ao executar, será instanciado um processo que ficará aceitando conexões HTTP na porta `8080`.

2.3 Como consumir o serviço usando o cURL

Para consumir o serviço REST implementado nessa seção você precisará de uma aplicação capaz de realizar requisições HTTP. O `curl` é um aplicativo de linha de comando presente no Linux que poderia ser usado para tal.

```
# Obtendo a mensagem padrão Olá mundo
curl http://localhost:8080/saudacao

# Obtendo a mensagem Olá Joao
curl -L -X GET 'http://localhost:8080/saudacao?nome=Joao'
```

Mas vale ver como consumir o serviço usando ferramentas mais completas, como o Bruno³ (software livre), o Postman⁴, Insomnia⁵ ou HTTPie⁶. O Visual Studio Code, com a extensão Bruno, também pode ser usado para consumir o serviço REST. O IntelliJ Ultimate também possui um cliente HTTP⁷ integrado que pode ser usado para consumir o serviço REST.

³<https://www.usebruno.com/>

⁴<https://www.getpostman.com/>

⁵<https://insomnia.rest>

⁶<https://httpie.io>

⁷<https://www.jetbrains.com/help/idea/http-client-in-product-code-editor.html>

3 Segundo exemplo: Verbos GET, POST, PUT e DELETE

Esse exemplo tem como único objetivo demonstrar como usar os verbos HTTP GET, POST, PUT e DELETE, bem como os códigos de estado do HTTP na resposta. O exemplo consiste de uma simples agenda de contatos armazenada em memória.

Tabela 2: API do Serviço Web Agenda de Contatos

Verbo	Recurso	Corpo do pedido	Corpo da resposta	HTTP Status
GET	/pessoas	–	Documento JSON com id, nome e email de todas pessoas	200
	/pessoas/{id}	–	Documento JSON com id, nome e email do id informado	200 ou 404
POST	/pessoas	Documento JSON com nome e email do novo contato	Documento JSON com id, nome e email	201
PUT	/pessoas	Documento JSON com novos valores para nome e email	Documento JSON com id, nome e email	200 or 404
DELETE	/pessoas/{id}	–	–	204 or 404

3.1 Crie os pacotes e as classes Java

Crie pacotes e classes Java para ficar de acordo com a estrutura apresentada abaixo:

```
.
|-- engtelecom
    |-- std
        |-- labrest
            |-- LabrestApplication.java
            |-- controller
            |   |-- AgendaController.java
            |   |-- SaudacaoController.java
            |-- entities
            |   |-- Pessoa.java
            |   |-- Saudacao.java
            |-- exceptions
            |   |-- PessoaNaoEncontradaException.java
            |-- service
            |   |-- PessoaService.java
```

Na Listagem 3 é apresentado o código da classe Pessoa.java. Trata-se de um *Plain Old Java Object* (POJO)⁸ para representar uma pessoa na agenda de contatos. É obrigatório ter um método construtor padrão (método sem parâmetros).

Listagem 3: Pessoa.java

```
package engtelecom.std.labrest.entities;

public class Pessoa {
    private long id;
    private String nome;
    private String email;

    public Pessoa() { }

    public Pessoa(String nome, String email) {
        this.nome = nome;
        this.email = email;
    }
}
```

⁸Você pode fazer uso do projeto Lombok para gerar o POJO de maneira mais simples. Veja <https://projectlombok.org/setup/gradle>

```

}

public long getId() {return id;}
public void setId(long id) {this.id = id;}
public String getNome() {return nome;}
public String getEmail() {return email;}
public void setNome(String nome) {this.nome = nome;}
public void setEmail(String email) {this.email = email;}
}

```

Na Listagem 4 é apresentado o código de uma classe que será usada caso o id da pessoa, informado pelo usuário, não estiver armazenado na memória.

Listagem 4: PessoaNaoEncontradaException.java

```

package engtelecom.std.labrest.exceptions;

public class PessoaNaoEncontradaException extends RuntimeException {

    public PessoaNaoEncontradaException(long id) {
        super("Não foi possível encontrar pessoa com o id: " + id);
    }
}

```

Na Listagem 5 é apresentado o conteúdo da classe PessoaService que neste exemplo representa um banco de dados, em memória, de objetos do tipo Pessoa.

Listagem 5: PessoaService.java

```

package engtelecom.std.labrest.service;

import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.atomic.AtomicLong;

import org.springframework.stereotype.Component;

import engtelecom.std.labrest.entities.Pessoa;

/**
 * PessoaService é uma classe que simula um banco de dados.
 *
 * A anotação @Component indica que a classe PessoaService é um componente do
 * Spring. Isso significa que o Spring irá gerenciar as instâncias dessa classe
 * e irá injetá-las onde for necessário.
 *
 * Classes anotadas com @Component são chamadas de beans. E são singleton por
 * padrão, ou seja, o Spring irá criar apenas uma instância dessa classe e irá
 * compartilhá-la entre todos os componentes que a utilizarem.
 *
 * https://java-design-patterns.com/patterns/singleton/
 */
@Component
public class PessoaService {

    // criando uma lista para simular um banco de dados em memória
    private List<Pessoa> pessoas;

    // criando um contador para gerar ids. O contador é estático para que seja
    // compartilhado entre todas as instâncias da classe PessoaService
    // O contador é do tipo AtomicLong para que as operações de incremento e
    // decremento sejam atômicas
    private static AtomicLong contador = new AtomicLong();

    public PessoaService() {
        pessoas = new ArrayList<>();
        // adicionando algumas pessoas para facilitar os testes
    }
}

```

```

        this.cadastrar(new Pessoa("João", "joao@example.org"));
        this.cadastrar(new Pessoa("Maria", "maria@example.net"));
        this.cadastrar(new Pessoa("Juca", "juca@example.com"));
    }

    public Pessoa cadastrar(Pessoa pessoa) {
        // gerando um id para a pessoa e ignorando o id enviado
        pessoa.setId(contador.incrementAndGet());
        pessoas.add(pessoa);
        return pessoa;
    }

    public List<Pessoa> buscarTodos() {
        return pessoas;
    }

    public Pessoa buscarPorId(Long id) {
        return this.pessoas.stream().filter(p -> p.getId().equals(id)).findFirst().orElse(null);
    }

    public Pessoa atualizar(Pessoa pessoa) {
        Pessoa p = buscarPorId(pessoa.getId());
        if (p != null) {
            p.setNome(pessoa.getNome());
            p.setEmail(pessoa.getEmail());
        }
        return p;
    }

    public boolean excluir(Long id) {
        return this.pessoas.removeIf(p -> p.getId().equals(id));
    }
}

```

Na Listagem 6 é apresentado o conteúdo do código da classe responsável por processar os pedidos a API REST e interagir com o banco de dados de pessoas. Cabe salientar que as práticas nesse projeto não são adequadas em um ambiente de produção, pois seria recomendado armazenar a agenda em um banco de dados em memória não volátil.

Listagem 6: AgendaController.java

```

package engtelecom.std.labrest.controller;

import java.util.List;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.ControllerAdvice;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.ResponseBody;
import org.springframework.web.bind.annotation.ResponseStatus;
import org.springframework.web.bind.annotation.RestController;

import engtelecom.std.labrest.entities.Pessoa;
import engtelecom.std.labrest.exceptions.PessoaNaoEncontradaException;
import engtelecom.std.labrest.service.PessoaService;

@RestController
// Mapeia as URLs /pessoas e /pessoas/ para esse controller

```

```

// Se desejar usar versionamento de API, basta adicionar a versão. Ex:
// /v1/pessoas
@RequestMapping({ "/pessoas", "/pessoas/" })
public class AgendaController {

    @Autowired // injeta uma instância de PessoaService que está anotada com @Component
    private PessoaService pessoaService;

    @GetMapping
    public List<Pessoa> obterTodasPessoas() {
        return this.pessoaService.buscarTodos();
    }

    @GetMapping("/{id}")
    @ResponseStatus(HttpStatus.OK)
    public Pessoa obterPessoa(@PathVariable long id) {

        Pessoa p = this.pessoaService.buscarPorId(id);

        if (p != null) {
            return p;
        }
        throw new PessoaNaoEncontradaException(id);
    }

    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    public Pessoa adicionarPessoa(@RequestBody Pessoa p) {
        return this.pessoaService.cadastrar(p);
    }

    @PutMapping
    @ResponseStatus(HttpStatus.OK)
    public Pessoa atualizarPessoa(@RequestBody Pessoa pessoa) {
        Pessoa p = this.pessoaService.atualizar(pessoa);
        if (p != null) {
            return p;
        }
        throw new PessoaNaoEncontradaException(pessoa.getId());
    }

    @DeleteMapping("/{id}")
    @ResponseStatus(HttpStatus.NO_CONTENT)
    public void excluirPessoa(@PathVariable long id) {
        if (!this.pessoaService.excluir(id)) {
            throw new PessoaNaoEncontradaException(id);
        }
    }

    @ControllerAdvice
    class PessoaNaoEncontrada {
        @ResponseBody
        @ExceptionHandler(PessoaNaoEncontradaException.class)
        @ResponseStatus(HttpStatus.NOT_FOUND)
        String pessoaNaoEncontrada(PessoaNaoEncontradaException p) {
            return p.getMessage();
        }
    }
}

```

3.2 Como consumir o serviço usando o cURL

```
# Obtendo a lista com todos os contatos
curl -L -X GET 'http://localhost:8080/pessoas/'

# Adicionando um novo contato
curl -L -X POST 'http://localhost:8080/pessoas' \
-H 'Content-Type: application/json' \
--data-raw '{
  "nome" : "Juca",
  "email": "juca@email.com"
}'

# Obtendo detalhes de um único contato com o id = 1
curl -L -X GET 'http://localhost:8080/pessoas/1'

# Alterando o email do contato com o id = 1
curl -L -X PUT 'http://localhost:8080/pessoas/1' \
-H 'Content-Type: application/json' \
--data-raw '{
  "nome" : "Juca",
  "email": "novojuca@email.com"
}'

# Excluindo o contato com o id = 1
curl -L -X DELETE 'http://localhost:8080/pessoas/1'
```

4 Aplicação Java para consumir serviço REST

Em Java existem diversos *frameworks* e bibliotecas para desenvolver ou consumir serviços REST. Nessa seção é apresentado um exemplo usando o OkHttp⁹, que é apenas um cliente HTTP em Java e que usaremos para consumir o serviço REST desenvolvido na Seção 3.

Crie um novo projeto Java com o *gradle* e deixe o conteúdo das seções *plugins* e *dependencies*, do arquivo `build.gradle`, igual ao quadro abaixo:

Listagem 7: `build.gradle` do Cliente REST

```
plugins {
    id 'application'
}

repositories {
    mavenCentral()
}

dependencies {
    // Cliente HTTP para Java
    // https://mvnrepository.com/artifact/com.squareup.okhttp3/okhttp
    implementation 'com.squareup.okhttp3:okhttp:5.3.2'

    // Para converter JSON em objetos Java e vice-versa
    // https://mvnrepository.com/artifact/com.google.code.gson/gson
    implementation 'com.google.code.gson:gson:2.14.0'

    // Para gerar dados aleatórios - https://www.datafaker.net/
    // https://mvnrepository.com/artifact/net.datafaker/datafaker
    implementation 'net.datafaker:datafaker:2.5.4'

    // Deixar as demais linhas já existentes nessa seção
}
```

⁹<https://square.github.io/okhttp/>

Crie um classe POJO (ou record) para representar uma Pessoa. Essa classe deve ter os seguintes atributos: Long id, String nome, String email.

```
package engtelecom.std;

public record Pessoa(Long id, String nome, String email) {}
```

Por fim, crie uma classe para fazer requisições GET e POST.

Listagem 8: Cliente REST

```
package engtelecom.std;

import com.google.gson.Gson;

import net.datafaker.Faker;
import okhttp3.HttpUrl;
import okhttp3.MediaType;
import okhttp3.OkHttpClient;
import okhttp3.Request;
import okhttp3.RequestBody;
import okhttp3.Response;

public class App {

    private OkHttpClient client = new OkHttpClient();
    private final String HOST = "http://localhost:8080/";
    private final MediaType MEDIA_TYPE = MediaType.get("application/json");

    public void listarTodas() throws Exception {

        // Padrao de projeto Builder
        // https://java-design-patterns.com/patterns/builder/

        // url = http://localhost:8080/pessoas
        HttpUrl url = HttpUrl.parse(HOST).newBuilder()
            .addPathSegment("pessoas")
            .build();

        Request request = new Request.Builder()
            .url(url)
            .get()
            .build();

        String response = client.newCall(request).execute().body().string();
        System.out.println(response);
    }

    public void cadastrarPessoa(Pessoa p) {
        HttpUrl url = HttpUrl.parse(HOST).newBuilder()
            .addPathSegment("pessoas")
            .build();

        RequestBody body = RequestBody.create(new Gson().toJson(p), MEDIA_TYPE);

        Request request = new Request.Builder()
            .url(url)
            .post(body)
            .build();

        try (Response response = client.newCall(request).execute()) {
            if (response.isSuccessful()) {
                System.out.println("Pessoa cadastrada com sucesso!");
            } else {
                System.out.println("Erro ao cadastrar pessoa!");
            }
        }
    }
}
```

```

    } catch (Exception e) {
        System.err.println("Erro ao cadastrar pessoa!");
    }
}

public void listarDadosDeUmaPessoa(Long id) throws Exception {
    // url = http://localhost:8080/pessoas/{id}
    HttpRequest url = HttpRequest.parse(HOST).newBuilder()
        .addPathSegment("pessoas")
        .addPathSegment(id.toString())
        .build();

    Request request = new Request.Builder()
        .url(url)
        .get()
        .build();

    String response = client.newCall(request).execute().body().string();
    System.out.println(response);
}

public static void main(String[] args) throws Exception {
    var app = new App();

    var faker = new Faker();// classe que gera dados aleatórios

    app.listarTodas();

    var nome = faker.name().firstName();
    var email = faker.internet().emailAddress();
    app.cadastrarPessoa(new Pessoa(null, nome, email));

    app.listarTodas();
    app.listarDadosDeUmaPessoa(1L);
}
}

```

5 Documentação de API com OpenAPI 3

A OpenAPI (<https://www.openapis.org>) é uma especificação que define uma linguagem padrão para especificar APIs HTTP (APIs REST). A especificação da API pode ser escrita em JSON ou YAML. A especificação OpenAPI é usada por diversas ferramentas para gerar código cliente e servidor, bem como para gerar documentação interativa da API (veja um exemplo em <https://petstore3.swagger.io>). Em <https://learn.openapis.org> encontrar um tutorial sobre a especificação OpenAPI.

A biblioteca `springdoc-openapi` (<https://springdoc.org>) é uma implementação da especificação OpenAPI para o framework Spring. Essa biblioteca pode ser usada para gerar a documentação da API REST desenvolvida nesse laboratório. Para isso, basta adicionar a dependência no arquivo `build.gradle` do projeto:

```

// https://mvnrepository.com/artifact/org.springdoc/springdoc-openapi-starter-webmvc-ui
implementation 'org.springdoc:springdoc-openapi-starter-webmvc-ui:2.8.14'

```

Após adicionar a dependência, acesse a URL `http://localhost:8080/swagger-ui.html` para visualizar a documentação da API REST desenvolvida nesse laboratório.

A documentação é gerada automaticamente a partir dos métodos anotados com `@GetMapping`, `@PostMapping`, etc. Caso queira baixar o arquivo JSON com a especificação da API, acesse a URL `http://localhost:8080/v3/api-docs` e caso queira baixar o arquivo YAML, acesse a URL `http://localhost:8080/v3/api-docs.yaml`.

Referências

1. <https://restfulapi.net/>
2. <https://spring.io/guides/gs/rest-service/>
3. <https://spring.io/guides/tutorials/rest/>
4. <https://docs.spring.io/spring-boot/docs/current/reference/html/application-properties.html>
5. <https://designer.mocky.io/>
6. <https://square.github.io/okhttp/recipes/>
7. <https://www.datafaker.net/>
8. <https://java-design-patterns.com>